A Non-dominated Sorting Biased Random-Key Genetic Algorithm for the Multi-Objective Physical Cell Identity Assignment Problem

Luis H. P. Mendes Fábio L. Usberti Mário C. San Felice
Carlos E. de Andrade

Institute of Computing – University of Campinas (UNICAMP)
Department of Computing – Federal University of São Carlos (UFSCar)
AT&T Labs Research

luis.mendes@ic.unicamp.br

16th Metaheuristics International Conference (MIC 2026)

Presentation Overview

1. Motivation and context
2. Problem definition
3. Proposed method
4. Experimental setup
5. Main computational results
6. Key takeaways
7. Conclusions and future work

Motivation and context

Why PCI planning is hard in practice

- **Every LTE/5G cell broadcasts a Physical Cell Identity (PCI)** so user equipment can detect and distinguish cells.
- **The PCI space is limited** (LTE: 504; 5G: 1008) $\Rightarrow$ reuse is unavoidable.
- Reuse must avoid **technological conflicts** (collision, confusion, and critical modulo-k interference).
- Operators also need **operational safety**: staged rollouts mean that a good final plan can still fail due to *PCI shuffles*.

We model this as a constrained multi-objective optimization problem and approximate its

Conflicts to avoid (technology + rollout)

- **Collision (hard):** neighbors share the same PCI $\rightarrow$ UE cannot distinguish signals.
- **Confusion (hard):** a cell has two distinct neighbors with the same PCI $\rightarrow$ ambiguous handover target.
- **Modulo-k collision:** neighbors have PCIs congruent modulo k $\rightarrow$ reference signals interfere (critical on *top neighbors*).
- **PCI shuffle:** batched rollout assigns a new PCI equal to a neighbor's *current* PCI $\rightarrow$ transient collision/confusion

Main contributions

1. We propose a **Non-dominated Sorting BRKGA (NS-BRKGA)** to solve the **multi-objective** PCI assignment problem with conflicting objectives.
2. We design a **random-key encoding + specialized decoder** that combines:
   - fast best-effort **constraint repair**,
   - **lexicographically guided** local search, and
   - **genotype feedback** (re-encoding improvements into chromosomes).
3. We benchmark on **291 real-world instances** against five representative MOEAs (incl. NSGA-II), showing **stronger Pareto fronts** under the same time budget.

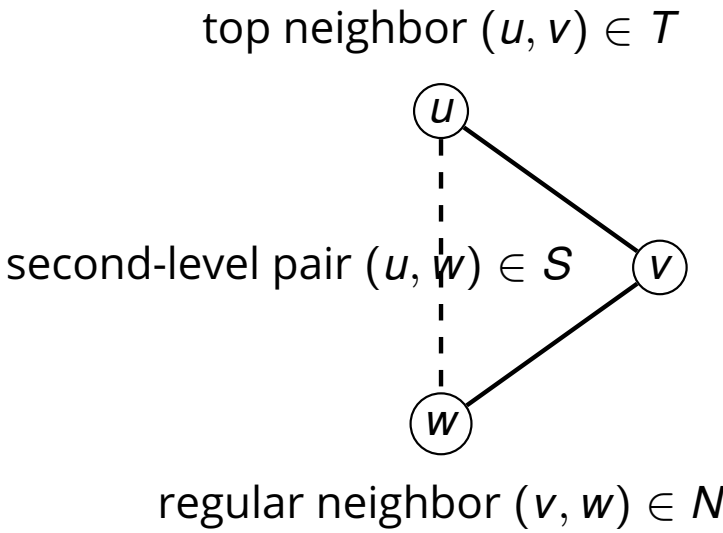
Problem definition

Network model and neighborhoods

We model the network as an undirected graph $G = (V, E)$:

- V: cells; E: neighbor/interference relations.
- The edge set is partitioned: $E = T \cup N, \ T \cap N = \emptyset$.
  - T: **top neighbors** (critical relations) where modulo-k collisions are *forbidden*;
  - N: regular neighbors where direct collisions are *forbidden*, while modulo-k collisions are *minimized*.
- Second-level neighbor pairs (used to model confusions):

$$S = \{(u, w) \mid (u, v) \in E, (v, w) \in E, (u, w) \notin E\}.$$



Dashed edges are *not* in E (they represent confusion pairs).

Decision variables and hard constraints

Decision variable: assign each cell $v \in V$ an integer PCI $x_v \in AP \subseteq \{0, 1, \ldots, PCI_{\max}\}$.

Hard constraints (must hold):

1. **Collision-freeness** (no adjacent equal PCIs):

$$x_u \neq x_v, \quad \forall (u, v) \in E.$$

2. **Top-neighbor modulo-k collision-freeness**:

$$x_u \not\equiv x_v \pmod{k}, \quad \forall (u, v) \in T.$$

3. **Confusion-freeness** (no two second-level neighbors share a PCI):

$$x_u \neq x_v, \quad \forall (u, v) \in S.$$

Three objectives (trade-offs)

Let $\mathbb{I}(\cdot)$ be the indicator function and O_v the original PCI of cell v.

1. **Modulo-k collisions on regular neighbors** (interference):

$$f_1(x) = \sum_{(u,v)\in N} \mathbb{I}\big(x_u \equiv x_v \pmod{k}\big).$$

2. **PCI shuffles** (deployment safety during batched rollouts):

$$f_2(x) = \sum_{(u,v)\in E\cup S} \Big(\mathbb{I}(x_u = O_v) + \mathbb{I}(x_v = O_u)\Big).$$

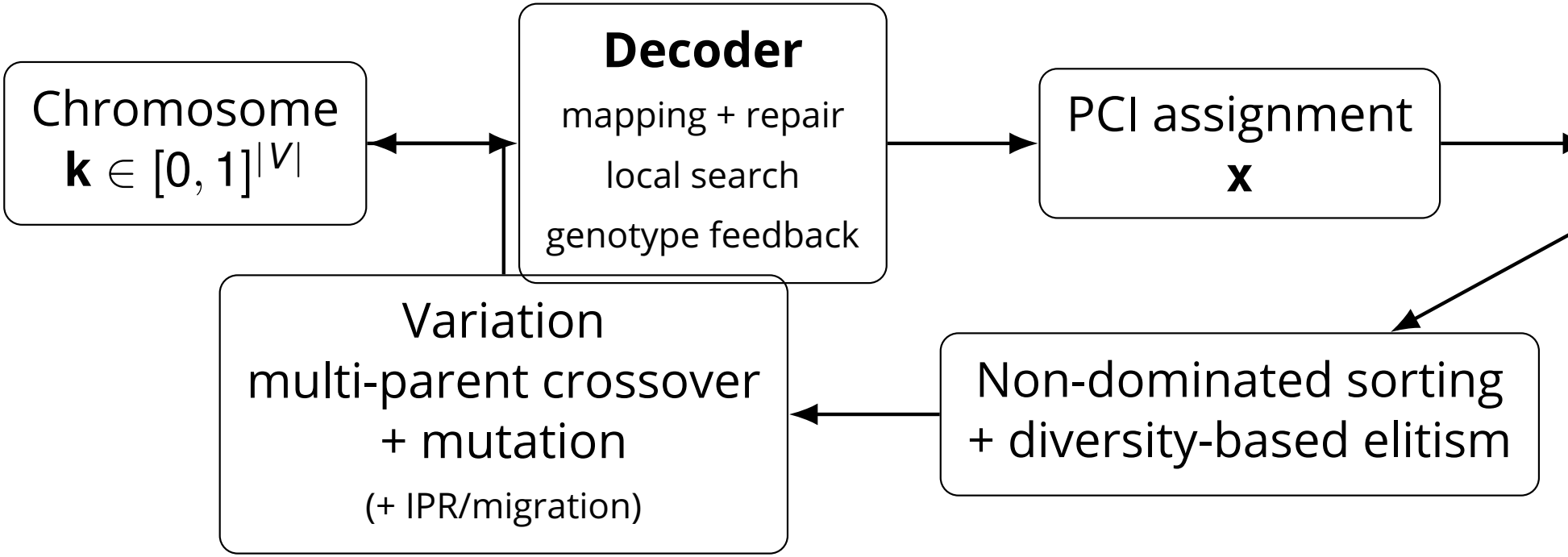
3. **Total PCI changes** (stability / renumbering cost):

$$f_3(x) = \sum_{v\in V} \mathbb{I}(x_v \neq O_v).$$

Goal: approximate the **Pareto front** for (f_1, f_2, f_3) over feasible assignments.

Proposed method (NS-BRKGA)

NS-BRKGA at a glance: random keys + memetic decoder



Key idea: a problem-agnostic evolutionary engine + a problem-specific decoder that enforces feasibility and intensifies the search.

Solution representation

A chromosome is a vector of random keys:

$$\mathbf{k} = (k_v)_{v \in V}, \qquad k_v \sim \mathcal{U}[0, 1].$$

Let AP be the sorted list of allowed PCIs, with $m = |AP|$.

Deterministic decoding (initial assignment):

$$x_v \;=\; AP[\lfloor k_v \cdot m \rfloor]\,.$$

- This decoding is fast, but it can yield **infeasible** assignments.
- Feasibility is enforced by an embedded repair + local improvement phase.

Decoder step 1: fast feasibility repair

We quantify hard-constraint violations via three counters:

$$v_{\text{col}}(\mathbf{x}) = \sum_{(u,v)\in E} \mathbb{I}(x_u = x_v),$$

$$v_{\text{tn}}(\mathbf{x}) = \sum_{(u,v)\in T} \mathbb{I}\big(x_u \equiv x_v \pmod{k}\big),$$

$$v_{\text{conf}}(\mathbf{x}) = \sum_{(u,v)\in S} \mathbb{I}(x_u = x_v).$$

A solution is feasible iff $v_{\text{col}} = v_{\text{tn}} = v_{\text{conf}} = 0$.

Greedy best-effort repair (high level):

- Build the set of *violated* cells $U \subseteq V$.
- Pick $v \in U$ with the most incident violations (tie: higher degree).
- Test all $p \in AP$ and choose $x_v \leftarrow p$ that minimizes $v_{\text{col}} + v_{\text{tn}} + v_{\text{conf}}$.
- Repeat while improving the total violation count.

Decoder step 1b: penalize residual infeasibility (so elites remain feasible)

If repair cannot remove all violations, we penalize the first objective:

$$\tilde{f}_1(\mathbf{x}) = f_1(\mathbf{x}) + M \cdot \big(v_{\mathrm{col}}(\mathbf{x}) + v_{\mathrm{tn}}(\mathbf{x}) + v_{\mathrm{conf}}(\mathbf{x})\big),$$

with a large constant M (set as $M = |N| + 1$ in the paper).

Returned fitness is:

$$\big(\tilde{f}_1(\mathbf{x}),\ f_2(\mathbf{x}),\ f_3(\mathbf{x})\big).$$

- Any remaining hard-constraint violation makes $\tilde{f}_1$ worse than the worst feasible f_1 value.
- This prevents infeasible solutions from dominating feasible ones.

Decoder step 2: lexicographically guided local search

Once feasible, we run local search over **feasible single-cell moves**:

$$x_v \leftarrow p, \quad p \in AP \setminus \{x_v\} \text{ and move preserves feasibility.}$$

Move acceptance (inside the decoder): first-improvement under a lexicographic order

$$(a_1, a_2, a_3) \prec (b_1, b_2, b_3) \;\Leftrightarrow\; \begin{cases} a_1 < b_1, \text{ or} \\ a_1 = b_1 \;\wedge\; a_2 < b_2, \text{ or} \\ a_1 = b_1 \;\wedge\; a_2 = b_2 \;\wedge\; a_3 < b_3. \end{cases}$$

- Local search prioritizes operational objectives in the order (f_1, f_2, f_3).
- Example (from the paper): increasing f_3 is accepted only if it improves f_2 or f_1.

Decoder step 3: genotype feedback (inherit local-search improvements)

After local search improves **x**, we **re-encode** the solution back into the chromosome.

If $x_v = AP[j]$ for $j \in \{0, \ldots, |AP| - 1\}$, set:

$$k_v \leftarrow \frac{j + 0.5}{|AP|} \quad \Rightarrow \quad \lfloor k_v \cdot |AP| \rfloor = j.$$

- The decoder becomes a **memetic operator**: improvements are not lost.
- Evolutionary operators can now recombine and propagate locally improved building blocks.

Evolutionary engine: NSGA-style ranking + BRKGA variation + intensification

- **Population ranking:** Pareto dominance via non-dominated sorting.
- **Elitism:** prioritize non-dominated solutions, then maximize **genotypic diversity** (distance in random-key space).
- **Variation:** biased **multi-parent crossover** ($\pi > 2$) + **polynomial mutation**.

Explicit intensification/diversification (paper configuration):

- Implicit Path-Relinking (IPR) every 500 generations.
- Multi-population scheme ($p = 3$) with migration every 200 generations ($k = 30$ elites migrated).
- Stagnation handling:
  - *shaking* after 200 stagnant generations,
  - partial *reset* after 500 stagnant generations.

Parameter settings used in the experiments (reference)

Parameter	**Value**
Population size $\|P\|$ (per subpopulation)	300
Parent count π (elitist π_e)	3 (with $\pi_e = 2$ elites)
Elite fraction $p^e_{\min}$ – $p^e_{\max}$	0.10 – 0.30
Bias function $\Phi(r)$	$1/\sqrt{r}$
Mutation probability p_m (index η_m)	0.01 ($\eta_m = 50$)
Number of subpopulations p	3
Migration interval (gen), size k	200 gen, $k = 30$ solutions
IPR interval (gen)	500 gen
Shaking trigger (stagnant gen)	200 gen ($p_{\text{shake}} = 0.33$, $\eta_{\text{shake}} = 20$)
Reset trigger (stagnant gen)	500 gen ($p_{\text{reset}} = 0.20$)

Experimental setup

Experimental setup (benchmark, baselines, metrics)

Benchmark and protocol:

- 291 real-world MOPCI instances (sizes from 30 to 4125 cells).
- Instance types (PCI groups):
  - `full`: $AP = \{0, \ldots, 1007\}$ (least constrained),
  - `orig`: $AP = \{\min(\mathbf{O}), \ldots, \max(\mathbf{O})\}$,
  - `squz`: minimal contiguous AP that still admits feasibility (most constrained).
- Budget: 15 minutes per run, 5 independent runs per instance.

Baselines (same time budget):

- NSGA-II, MOEA/D-DE, NSPSO, MHACO, IHS.

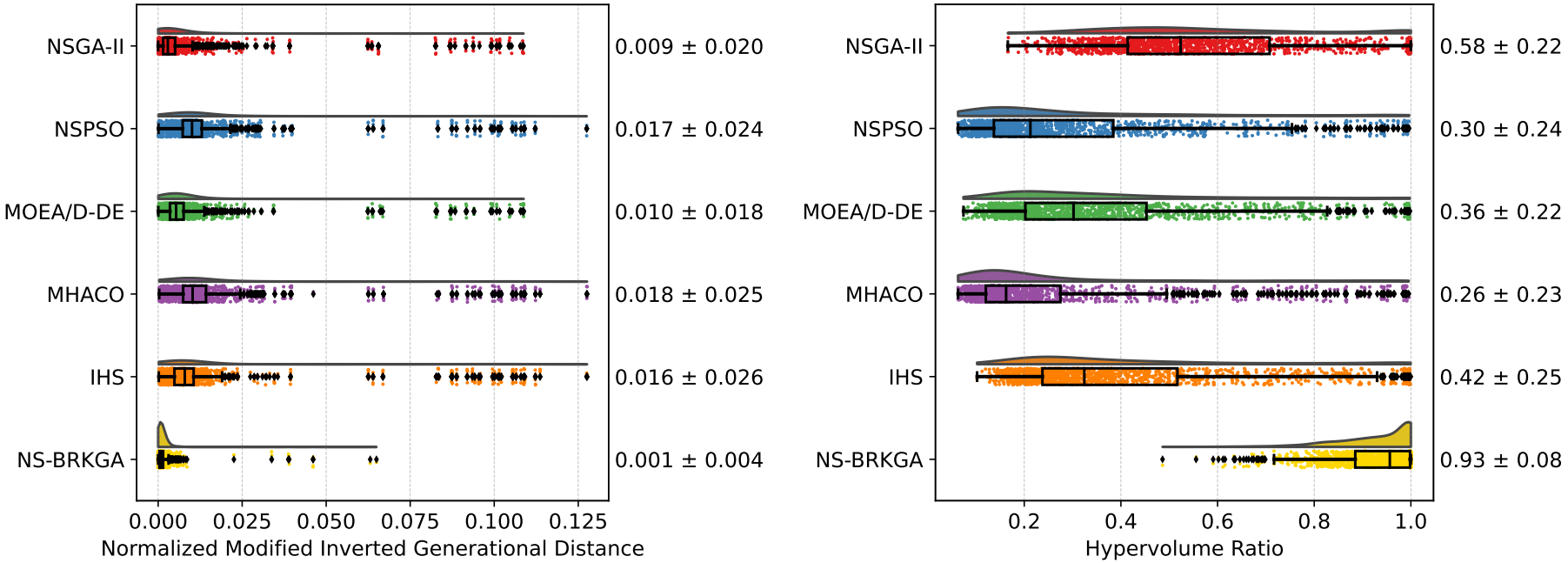
Indicators (per instance):

- $NIGD^+$ (lower is better): convergence to a reference front.
- HVR (higher is better): dominated hypervolume ratio vs the reference front.

Reference front: pooled non-dominated set across all algorithms and runs (filtered

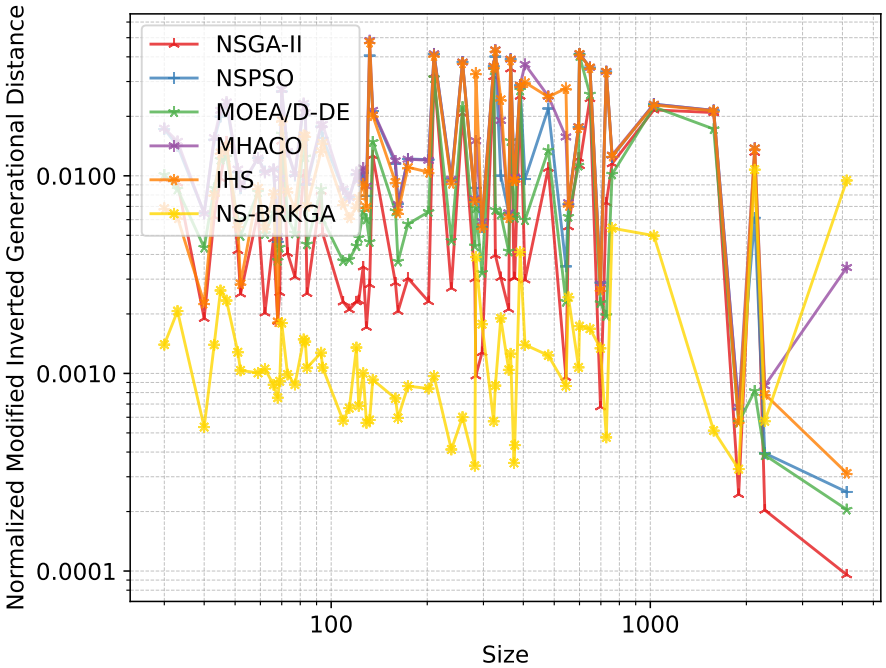
Main computational results

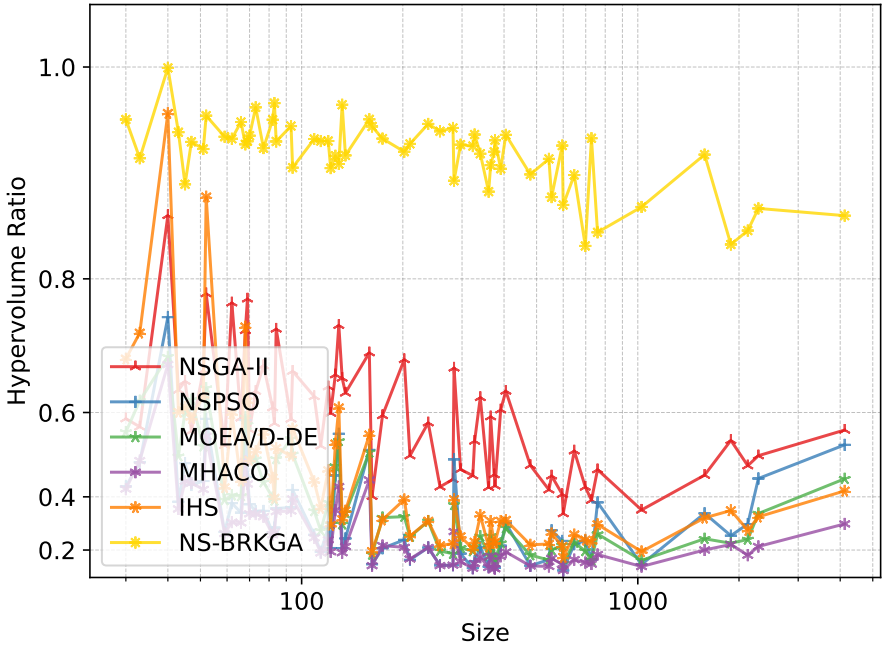
Overall performance across all 291 instances



- NS-BRKGA attains the best mean ranks on **both** $NIGD^+$ and HVR.
- Reported means: $NIGD^+ \approx 0.001$ and $HVR \approx 0.93$ (NSGA-II: 0.009 and 0.58).

Scalability with problem size





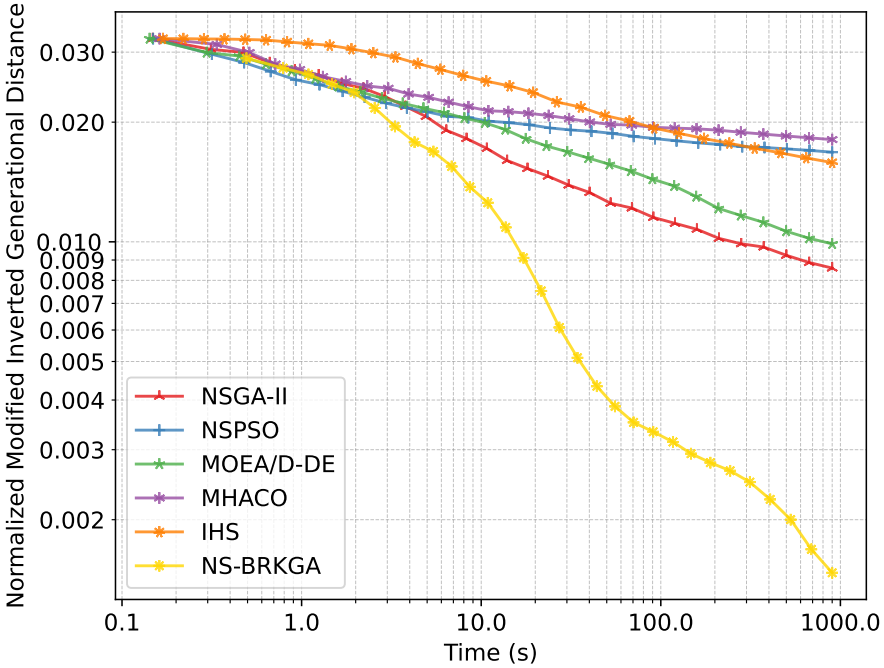
- As $|V|$ grows, baseline methods degrade markedly, while NS-BRKGA remains stable.
- On the largest instances ($|V| = 4125$), NS-BRKGA maintains mean $HVR \approx 0.87 \pm 0.08$ (NSGA-II $\approx 0.6 \pm 0.1$).

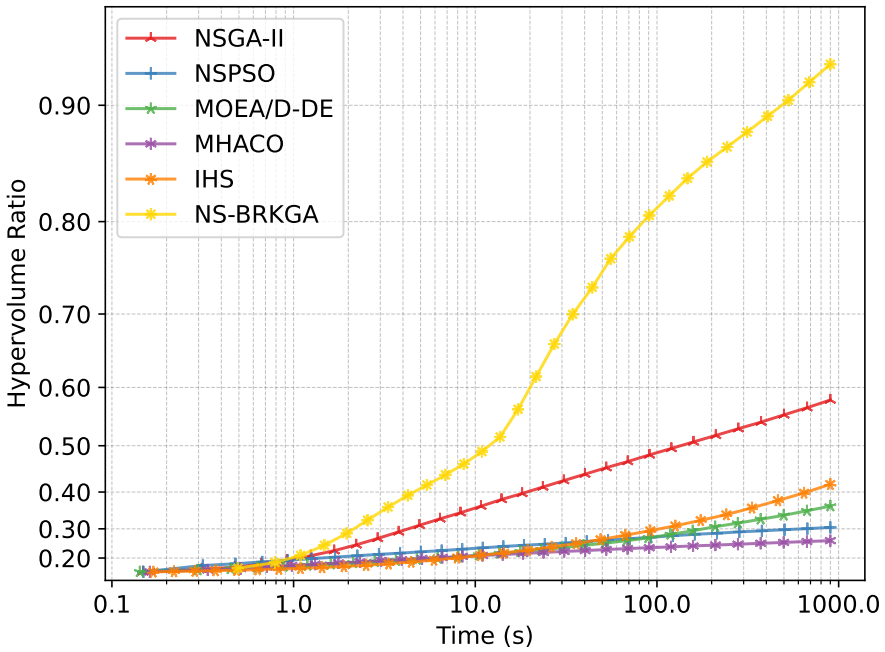
Robustness across instance types (constraint tightness)

Algorithm	*NIGD*$^+$ (↓)			*HVR* (↑)		
	`full`	`orig`	`squz`	`full`	`orig`	`squz`
NSGA-II	0.005 ± 0.006	0.004 ± 0.004	0.017 ± 0.032	0.46 ± 0.17	0.51 ± 0.11	0.77 ± 0.23
NSPSO	0.011 ± 0.007	0.011 ± 0.006	0.029 ± 0.037	0.22 ± 0.14	0.20 ± 0.11	0.49 ± 0.30
MOEA/D-DE	0.007 ± 0.005	0.006 ± 0.004	0.017 ± 0.030	0.27 ± 0.12	0.30 ± 0.13	0.53 ± 0.27
MHACO	0.011 ± 0.007	0.011 ± 0.007	0.033 ± 0.039	0.15 ± 0.07	0.17 ± 0.09	0.47 ± 0.30
IHS	0.008 ± 0.005	0.007 ± 0.004	0.033 ± 0.040	0.33 ± 0.17	0.36 ± 0.16	0.57 ± 0.30
NS-BRKGA	$\mathbf{0.001 \pm 0.001}$	$\mathbf{0.001 \pm 0.001}$	$\mathbf{0.002 \pm 0.007}$	$\mathbf{0.90 \pm 0.09}$	$\mathbf{0.91 \pm 0.08}$	$\mathbf{0.97 \pm 0.05}$

- NS-BRKGA is competitive on `full`/`orig` and decisively best on `squz`.
- On `squz`: mean *HVR* ≈ 0.97 and mean *NIGD*$^+$ ≈ 0.002.

Runtime behavior: strong early convergence





- NS-BRKGA reaches high-quality fronts **very early** (paper reports within $\sim$ 30s: $HVR \approx 0.7 \pm 0.2$).
- Baselines often do not match this quality even after the full 15-minute budget.

Key takeaways

What to remember

- The MOPCI is a large-scale **constrained** MOP requiring trade-offs among interference, deployment safety, and stability.
- NS-BRKGA's advantage comes from a **memetic decoder**:
  - fast feasibility repair,
  - lexicographic local search reflecting operational priorities,
  - genotype feedback to inherit improvements.
- On 291 real-world instances, NS-BRKGA yields **better Pareto fronts** than five MOEA baselines under the same time budget, and scales to thousands of cells.

Conclusions and future work

Conclusions and future work

- We introduced a memetic NS-BRKGA for the multi-objective PCI assignment problem, combining a problem-agnostic evolutionary engine with a problem-specific decoder.
- The method provides a robust way to navigate trade-offs in LTE/5G network planning, showing strong performance, scalability, and robustness on highly constrained instances.
- Future work mentioned in the paper:
  - exact-solver comparisons on smaller instances (MIP formulation exists but is prohibitive at scale),
  - extensions to richer network models (e.g., heterogeneous multi-layer networks),
  - incorporation of additional operational/technological constraints when needed.

Thank you!

Questions?

luis.mendes@ic.unicamp.br

(Code and benchmark links are provided in the paper.)